

Web Development & MERN Roadmap

Comprehensive Curriculum: Pre-React Foundations through Full-Stack & Mobile Deployment

Target Stack: MERN (Mongo, Express, React, Node)

Focus App: Business Inventory System

Approach: Build-As-You-Learn

What Was Added Beyond the Initial "Basics"?

In addition to standard HTML, CSS, and JS syntax, professional development requires bridge skills: **Web Infrastructure & Tools** (Git, Terminal, DevTools), **Data Integration** (Fetch API, Async/Await, Web Storage), and **Full-Stack Architecture** (Node, Express, Mongo, Authentication).

PHASE 1: DEVELOPER ENVIRONMENT & INFRASTRUCTURE

Essential setup and web mechanics before writing application logic.

[]	Command Line / Terminal	ADDED	Navigating directories (<code>cd</code> , <code>ls / dir</code>), creating files, running Node/npm scripts.
[]	Git & GitHub	ADDED	Version control basics (<code>git init</code> , <code>add</code> , <code>commit</code> , <code>push</code> , <code>branch</code>). Essential for backup and deployment.
[]	How the Web Works	ADDED	Client vs. Server architecture, HTTP/HTTPS request-response cycle, status codes (200, 404, 500).
[]	Browser DevTools	ADDED	Inspecting DOM elements, debugging JS errors in Console, monitoring Network requests.

PHASE 2: HTML5 — DOCUMENT STRUCTURE & FORMS

[]	Skeleton & Semantics		<code><!DOCTYPE></code> , <code><html></code> , <code><head></code> , <code><body></code> , <code><header></code> , <code><nav></code> , <code><main></code> , <code><section></code> , <code><footer></code> .
[]	Typography & Lists		Headings (<code><h1></code> - <code><h6></code>), paragraphs, spans, links (<code><a></code>), unordered (<code></code>) and ordered (<code></code>) lists.
[]	Forms & Controls	CRITICAL	<code><form></code> , <code><input></code> (text, number, date, checkbox), <code><select></code> , <code><option></code> , <code><textarea></code> , <code><button></code> . Attributes: <code>name</code> , <code>value</code> , <code>placeholder</code> , <code>type</code> .
[]	Tables		<code><table></code> , <code><thead></code> , <code><tbody></code> , <code><tr></code> , <code><th></code> , <code><td></code> . Used for displaying structured stock lists.
[]	DOM Attributes	ADDED	<code>id</code> , <code>class</code> , custom <code>data-*</code> attributes for binding JS data to HTML elements.

BUILD MILESTONE 1: Create an HTML page with an inventory form (Item, SKU, Quantity) and a table with hardcoded items.

PHASE 3: MODERN CSS3 — LAYOUTS & DESIGN SYSTEMS

[]	Box Model & Resets	<code>margin</code> , <code>border</code> , <code>padding</code> , <code>content</code> . Global reset with <code>box-sizing: border-box;</code> .
[]	Selectors & Cascade	Classes (<code>.btn</code>), IDs (<code>#main</code>), element selectors, pseudo-classes (<code>:hover</code> , <code>:focus</code>). Specificity rules.
[]	Flexbox Layouts CRITICAL	<code>display: flex</code> , <code>flex-direction</code> , <code>justify-content</code> , <code>align-items</code> , <code>gap</code> . (Navbars, toolbars, form alignment).
[]	CSS Grid Layouts	<code>display: grid</code> , <code>grid-template-columns</code> (<code>repeat</code> , <code>fr</code> , <code>minmax</code>), <code>gap</code> . (Dashboard multi-column cards).
[]	Positioning & Modals	<code>position: relative</code> , <code>absolute</code> , <code>fixed</code> , <code>sticky</code> , <code>z-index</code> . (Sticky headers, popup dialogs).
[]	Responsive Design ADDED	Media queries (<code>@media (max-width: 768px)</code>), mobile-first layout rules.
[]	CSS Variables ADDED	Declaring <code>:root { --primary: #3182ce; }</code> for central theme colors.

BUILD MILESTONE 2: Style the inventory HTML into a modern dashboard layout using Flexbox/Grid that collapses neatly on mobile screens.

PHASE 4: JAVASCRIPT (ES6+) — CORE LOGIC & DOM MANIPULATION

[]	Variables & Types	<code>const</code> and <code>let</code> (avoid <code>var</code>). Strings, numbers, booleans, arrays, objects, <code>null</code> , <code>undefined</code> .
[]	Functions & Logic	Arrow functions (<code>() => {}</code>), default parameters, <code>if/else</code> , Ternary operators (<code>cond ? a : b</code>).
[]	Array Methods CRITICAL	<code>.map()</code> (render lists), <code>.filter()</code> (delete/search), <code>.reduce()</code> (sum stock value), <code>.find()</code> .
[]	Objects & Immutability	Destructuring (<code>const { name, qty } = item</code>), Spread operator (<code>...</code>) for copying/ updating arrays without mutation.
[]	DOM & Events	<code>document.querySelector()</code> , <code>addEventListener('submit')</code> , <code>e.preventDefault()</code> , updating dynamic DOM nodes.
[]	Web Storage API ADDED	<code>localStorage.setItem()</code> / <code>getItem()</code> with <code>JSON.stringify()</code> / <code>JSON.parse()</code> to persist browser data.
[]	Async & Fetch API ADDED	Promises, <code>async/await</code> , <code>fetch()</code> for making HTTP API calls.
[]	ES6 Modules ADDED	Modular code splitting using <code>export const helper = ...</code> and <code>import { helper } from './file.js'</code> .

BUILD MILESTONE 3 (VANILLA JS CAPSTONE): Connect JS to your dashboard. Allow adding, deleting, searching stock, calculating total quantity, and saving items to `localStorage`.

PHASE 5: REACT — FRONT-END APPLICATION FRAMEWORK

[]	Tooling & JSX	Project setup with Vite (<code>npm create vite@latest</code>), JSX syntax rules, embedded JS <code>{expression}</code> .
[]	Components & Props	Building modular components (<code><InventoryTable /></code> , <code><AddItemForm /></code>), passing dynamic data with <code>props</code> .
[]	State (<code>useState</code>)	Managing UI state, updating state immutably using the spread operator (<code>[...items, newItem]</code>).
[]	Effects (<code>useEffect</code>)	Handling side effects: loading initial stock data on mount, syncing with external APIs.
[]	Controlled Forms ADDED	Binding form input <code>value</code> to React state and handling input changes with <code>onChange</code> .
[]	React Router ADDED	Multi-page routing with <code><BrowserRouter></code> , <code><Routes></code> , <code><Route></code> , <code><Link></code> (Dashboard vs. Settings views).

BUILD MILESTONE 4: Refactor your Vanilla JS inventory app into a React Single Page App broken down into clean, reusable components.

PHASE 6: THE NODE.JS & EXPRESS BACKEND

[]	Node Environment	Running JS on server, package management with NPM (<code>npm init</code> , <code>npm install express cors dotenv</code>).
[]	Express REST API	Building HTTP routes: <code>GET /api/items</code> , <code>POST /api/items</code> , <code>PUT /api/items/:id</code> , <code>DELETE /api/items/:id</code> .
[]	Middleware ADDED	<code>express.json()</code> request body parsing, CORS headers, error handling middleware.
[]	Environment Vars ADDED	Configuring <code>dotenv</code> (<code>.env</code>) to hide sensitive API keys and database credentials.

PHASE 7: MONGODB & MONGOOSE DATABASE LAYER

[]	NoSQL Concepts	Collections, Documents, JSON-like document structure, MongoDB Atlas cloud setup.
[]	Mongoose ORM	Defining Schemas (<code>name: String</code> , <code>qty: Number</code>), models, and field validation rules.
[]	CRUD Operations	<code>Item.find()</code> , <code>Item.create()</code> , <code>Item.findByIdAndUpdate()</code> , <code>Item.findByIdAndDelete()</code> .
[]	Authentication ADDED	User signup/login with password hashing (<code>bcrypt</code>) and JWT tokens to isolate data per account.

BUILD MILESTONE 5 (FULL MERN STACK APP): Connect your React front-end to Node/Express & MongoDB. Stock data now persists permanently in the cloud.

PHASE 8: MOBILE & DESKTOP EXPANSION (TARGET GOAL)

[]	React Native ADDED	Building native mobile UI (<code><View></code> , <code><Text></code>). Use phone cameras for inventory barcode scanning.
[]	Electron / Tauri ADDED	Wrapping web app into standalone desktop executables (<code>.exe</code> , <code>.dmg</code>) for Windows & macOS.
[]	Deployment ADDED	Hosting front-end on Vercel/Netlify, back-end API on Render, database on MongoDB Atlas.